Data Structure and Abstractions: Homework 5

Due on Tuesday, March 3, 2026, 11:59 pm (Thai time)

Kanat Tangwongsan Section 2

Jiraroj Wiruchpongsanon (6781617)

Data Struct. and {Abstractions, OOP} (Term II/2025–26)
built on 2026/02/24 at 12:23:04
due: tue mar 03 @ 11:59pm

Important Notes from the Assignment

This assignment contains only written questions focusing on performance characterization. You must typeset your answer, produce a PDF file called `hw5.pdf`, and upload it to Canvas. No other format will be accepted. To typeset your homework, apart from Microsoft Word, there are LibreOffice and LaTeX, which we recommend. Note that a scan of your handwritten solution will not be accepted. You can find helpful resources about how to typeset your work on the course website.

Collaboration

We interpret collaboration very liberally. You may work with other students. However, each student must write up and hand in his or her assignment separately. Let us repeat: You need to write your own code. You must not look at or copy someone else's code. You need to write up answers to written problems individually. The fact that you can recreate the solution from memory will be taken as proof that you actually understood it, and you may actually be interviewed about your answers.

Task 1: Hello, Definition (4 points)

This task consists of small problems involving working directly with the definition of Big-O. (Hint: One very simple solution for the first two problems below involves just taking limits.)

1. Show, using either definition, that $f(n) = n$ is $O(n \log n)$.
2. In class, we saw that Big-O multiplies naturally. You will explore this more formally here. Prove the following statement mathematically (i.e. using proof techniques learned in Discrete Math):

   Proposition: If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then the product $d(n)e(n)$ is $O(f(n)g(n))$.
3. There is a function `void fnE(int i, int num)` that runs in $1000 \cdot i$ steps, regardless of what `num` is. Consider the following code snippet:

   ```
   void fnA(int S[]) {
       int n = S.length;
       for (int i=0;i<n;i++) {
           fnE(i, S[i]);
       }
   }
   ```

   What's the running time in Big-O of `fnA` as a function of n, which is the length of the array S. You should assume that it takes constant time to determine the length of an array.
4. Show that $h(n) = 16n^2 + 11n^4 + 0.1n^5$ is not $O(n^4)$.

Solution

1. find the limit of $f(n)$ and $g(n)$

$$\lim_{n\to\infty} \frac{n}{n \log n} = \lim_{n\to\infty} \frac{1}{\log n} = 0 < \infty$$

   which means there exist a constant $c > 0$, such that for every positive integer n, $f(n) \leq c \cdot g(n)$

2. *Proof.* from the definition of big-Oh, we can write $d(n)$ as

$$d(n) = c_1 f(n)$$

   and $e(n)$ as

$$e(n) = c_2 g(n)$$

   so, to check whether $d(n)e(n) \in O(f(n)g(n))$ we do

$$\begin{aligned} \lim_{n\to\infty} \frac{d(n)e(n)}{f(n)g(n)} &= \lim_{n\to\infty} \frac{c_1 f(n) c_2 g(n)}{f(n)g(n)} \\ &= c_1 c_2 < \infty \end{aligned}$$

   □

3. if we write that out, it'll be

$$1000(0) + 1000(1) + 1000(2) + ... + 1000(n)$$

that's

$$1000 \sum_{i=0}^{n} i = 1000 \frac{n(n+1)}{2}$$

which is $\boxed{O(n^2)}$.

4. there's no constant c that satisfy $h(n) \leq c \cdot n^4$ when $h(n) = 16n^2 + 11n^4 + 0.1n^5$, to prove this, we find limit of $h(n)$ by n^3 as

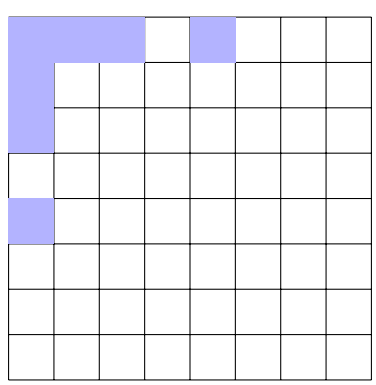
$$\begin{aligned} \lim_{n\to\infty} \frac{h(n)}{n^4} &= \lim_{n\to\infty} \frac{0 + 11 + 0.1n}{1} && \text{divide both numerator and denominator with } n^4 \\ &= \infty \not< \infty \end{aligned}$$

Task 2: Poisoned Wine (2 points)

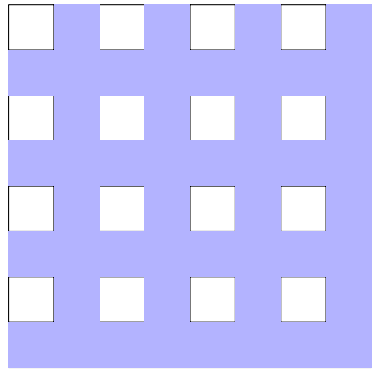
You own n bottles of wine, exactly one of which has been poisoned. You don't know which bottle, however. What you know is, if someone drinks just a tiny amount of wine from the poisoned bottle, s/he will start laughing uncontrollably after 30 days. In fact, the poison is so potent that even the faintest drop, diluted over and over, will still cause the symptom.
Design a scheme that determines exactly which one bottle was poisoned. You are allowed 31 days and can expend $O(\log n)$ testers (people). *Explain why your scheme meets the $O(\log n)$-tester requirement.*
(Hint: Numbers between 1 and n (inclusive) can be represented using $\lceil \log_2(n+1) \rceil$ bits.)
(More Hint: Think hard about the first hint before being spoiled by this hint. If the bottles are labeled 1 through n, how can we find out the i-th bit of the label of the poisoned bottle?)

Solution

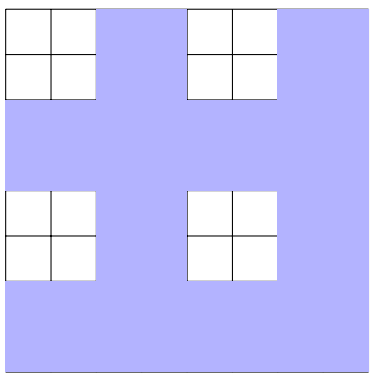
use the idea from humming code, put the bottles in the smallest $2^n \cdot 2^n$ grid form, it's fine even if the grid isn't completely filled. we do something like a binary search, but with rows or column of 'thickness' 2^i when $0 \leq i \leq (n-1)$, like a sieve. the image right below is all the entry used for the extended humming code, for 2^0, 2^1 and 2^2. the entry (1, 1) is for minor error correction that works to some extent.

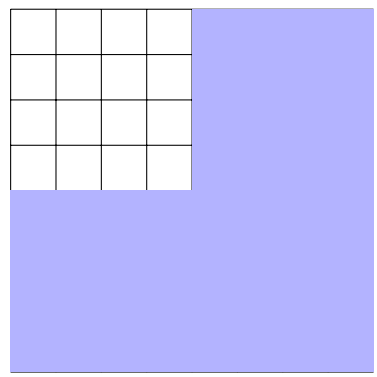


for rows and column with 2^0, start the iteration by skipping the first chunk (of length 2^i), pick the next one, then skip, and pick ... until we fill the length of the grid. we use two testers, one for all the rows and nother one for the column, as



and for thickness of 2^1 (left) and 2^2 (right)





so, we should need only $2 \cdot n$ testers for $2^n \cdot 2^n$ bottles, and after we got all the results (at least 2 unfortunate testers), there should be a logical way to figure out the exact entry that contain the poison.

now, convert this exponential scaling to a linear one, let $m = 2^{n+1} \rightarrow n = \log_2 m - 1$; thus for m bottles, we'll need $2(\log_2 m - 1) = 2\log_2 m - 2$ testers, which $2\log_2 m - 2 \in O(\log_2 m)$

Task 3: How Long Does This Take? (2 points)

For each of the following functions, determine the running time in terms of Θ in the variable n. Show your work. We're more interested in the thought process than the final answer.

```
void programA(int n) {
    long prod = 1;
    for (int c=n;c>0;c=c/2)
        prod = prod * c;
}

void programB(int n) {
    long prod = 1;
    for (int c=1;c<n;c=c*3)
```

```
        prod = prod * c;
}
```

Solution

for ‘programA‘, the function accepts parameter ‘n‘, we ignore ‘prod‘ and the operation inside the loop, as multiplication is the result of the loop’s condition, and multiplication usually be done in a single instructions/ clock cycles. so, we focus on the loop’s condition itself

the loop exits when ‘c‘ ≤ 0, which initially, ‘c‘ = ‘n‘, then in each iteration, c got divided by 2, so ‘c‘’s value would be in a sequence of

$$c, \frac{c}{2^1}, \frac{c}{2^2}, \frac{c}{2^3}, ..., \frac{c}{2^k}$$

which

$$\frac{c}{2^k} = 1$$

or

$$2^k = c \text{ when ‘c‘ = ‘n‘}$$

therefore,

$$k = \log_2 c = \log_2 n$$

and we can conclude that $\boxed{\text{‘programA‘} \in \Theta(\log n)}$

---

for ‘programB’, just like the previous one, we only focus on the loop’s condition itself.

as ‘c‘ initially starts at 1, tripled it’s value in every iteration, and the loop terminates when ‘c‘ $\geq$ ‘n‘, we can write the ‘c‘’s value sequence as

$$1, 1 \cdot 3^1, 1 \cdot 3^2, 1 \cdot 3^3, ..., 1 \cdot 3^k$$

which

$$3^k \geq n$$

therefore,

$$k = \log_3 n$$

and we can conclude that $\boxed{\text{‘programB‘} \in \Theta(\log_3 n)}$

Task 4: Halving Sum (4 points)

Our course staff has a signature process for summing up the values in a sequence (i.e., array). Let X be an input sequence consisting of n floating-point numbers. To make life easy, we'll assume n is a power of two—that is, $n = 2^k$ for some nonnegative integer k. To sum up these numbers, we use the following process, expressed in a Python-like language:

```
def hsum(X): # assume len(X) is a power of two
    while len(X) > 1:
        (1) allocate Y as an array of length len(X)/2
        (2) fill in Y so that Y[i] = X[2i] + X[2i+1] for i = 0, 1, ..., len(X )/2 - 1
        (3) X = Y
    return X[0]
```

This task has two parts:
Part I: (2 points) Observe that the amount of work done in Steps (1)–(3) is a function of the length of X in that iteration. If $z = X.length$ at the start of an iteration, how much work is being done in that iteration as a function z (e.g., $10z^5 + z \log z$ or $k_1 z^2 + k_2 z$ for some $k_1, k_2 \in \mathbb{R}^+$)? Don't use Big-O; answer in terms of k_1 and k_2. Let's make some assumptions here. For some $k_1, k_2 \in \mathbb{R}^+$:

- Allocating an array of length z costs you $k_1 \cdot z$.
- Arithmetic operations, as well as reading a value from an array and writing a value to an array, can be done in k_2 per operation.

Your answer for this part should look like the following: $(\cdot)z + (\cdot)$.
Part II: (2 points) Then, you'll analyze the running time of the algorithm (remember to explain how you get the running time you claim). To help you get started, make a table of how $X.length$ changes over time if we start with X of length, say, 64. How does this work in general? (Hint: The geometric sum formula presented in class may come in handy.)

Solution

1. observe operations inside the loop, (1) costs $k_1 \cdot \frac{z}{2}$,

   then (2) would cost

   - read $X[2i]$ — k_2 operation
   - read $X[2i+1]$ — k_2 operation
   - add $X[2i]$ and $X[2i+1]$ — k_2 operation
   - save the result to $Y[i]$ — k_2 operation

   that's $4k_2$ operations in total for each of Y's entry, so $4k_2 \cdot \frac{z}{2} = 2k_2 z$

   and for (3), this can vary (depend on the language, as far as i know, in some language like c, you can just assign X as some pointer address to Y, but in java, the language will try to copy new data every time (like in case of 'String', that's why we use 'StringBuilder')); best case (in terms of least operations), we only have to assign new addresss (write) to X, so that's k_2 operation; and for the worse case, we copy the whole thing, so just like (1), it's $k_1 \cdot \frac{z}{2}$

   as these operation are not sequencial, we just add them up, as

$$\boxed{\left(\frac{k_1}{2} + 2k_2\right) z + k_2}$$

for best case, and

$$\boxed{(k_1 + 2k_2)\, z}$$

in worse case.

---

2. as the while loop terminates when ‘len(X)‘ ≤ 0, and the in-loop operation do change X’s size to ‘len(X)/2‘ in every iteration, it’s a decreasing sequence, and the loop will terminate at some point. we try with a table of ‘X.length‘-iterations first, with ‘X.length‘ = 64:

Iteration	X**.length (base 10)**	X**.length (2^k form)**
0	64	2^6
1	32	2^5
2	16	2^4
3	8	2^3
4	4	2^2
5	2	2^1
6	1	2^0

as the given code is written in python, i’ll assume python do it’s magic, and will use the best case time per iteration of $\left(\frac{k_1}{2} + 2k_2\right) z + k_2$, where $z =$ ‘X.length‘, so the total time usage would be

$$\sum_{j=0}^{\log_2 n - 1} \left[\left(\frac{k_1}{2} + 2k_2 \right) \frac{n}{2^j} + k_2 \right]$$

which is equivalent to

$$\begin{aligned} T(n) &= \left(\frac{k_1}{2} + 2k_2 \right) \sum_{j=0}^{\log_2 n - 1} \frac{n}{2^j} + k_2 \sum_{j=0}^{\log_2 n - 1} 1 \\ &= \left(\frac{k_1}{2} + 2k_2 \right) n \sum_{j=0}^{\log_2 n - 1} \frac{1}{2^j} + k_2 \log_2 n \end{aligned}$$

for the geometric series,

$$\sum_{j=0}^{\log_2 n - 1} \frac{1}{2^j} = 2\left(1 - \frac{1}{n}\right)$$

so we get

$$T(n) = \left(\frac{k_1}{2} + 2k_2 \right) n \cdot 2 \left(1 - \frac{1}{n}\right) + k_2 \log_2 n$$

$$= (k_1 + 4k_2)(n-1) + k_2 \log_2 n \in \boxed{\Theta(n)}$$

Task 5: More Running Time Analysis (6 points)

For the most part, we have focused almost exclusively on worst-case running time. In this problem, we are going to pay closer attention to these Java methods and consider their worst-case and best-case behaviors. The best-case behavior is the running time on the input that yields the fastest running time. The worst-case behavior is the running time on the input that yields the slowest running time.

(1) Determine the best-case running time and the worst-case running time of `method1` in terms of Θ.

```
static void method1(int[] array) {
    int n = array.length;
    for (int index=0;index<n-1;index++) {
        int marker = helperMethod1(array, index, n - 1);
        swap(array, marker, index);
    }
}

static void swap(int[] array, int i, int j) {
    int temp=array[i];
    array[i]=array[j];
    array[j]=temp;
}

static int helperMethod1(int[] array, int first, int last) {
    int max = array[first];
    int indexOfMax = first;
    for (int i=last;i>first;i--) {
        if (array[i] > max) {
            max = array[i];
            indexOfMax = i;
        }
    }
    return indexOfMax;
}
```

(2) Determine the best-case running time and the worst-case running time of `method2` in terms of Θ.

```
static boolean method2(int[] array, int key) {
    int n = array.length;
    for (int index=0;index<n;index++) {
        if (array[index] == key) return true;
    }
    return false;
}
```

(3)
Determine the best-case running time and the worst-case running time of `method3` in terms of Θ.

```
static double method3(int[] array) {
    int n = array.length;
    double sum = 0;
```

```
4     for (int pass=100; pass >= 4; pass--) {
5         for (int index=0;index < 2*n;index++) {
6             for (int count=4*n;count>0;count/=2)
7                 sum += 1.0*array[index/2]/count;
8         }
9     }
10    return sum;
11 }
```

Solution

Remark. I assign line numbers to the code, which i will referenced from — line 67 as (67). line numbers will get reset in every sub-task.

1. firstly, on (2), ‘array.length‘ should be $\Theta(1)$, as the ‘array‘ object got ‘length‘ variable as static type.

   then, we enter the loop, the loop just goes through every index of the array, starting from index 0. inside the loop, (4), ‘helperMethod1‘ was called, which the method goes through the whole range of the array, starting from ‘index‘ this time — which ‘index‘’s value will change in every iteration of the loop. the ‘helperMethod1‘ returns the index of max value in the array from the given range — which would cost

   $$n, n-1, n-2, n-3, ..., 2, 1$$

   as ‘index‘ goes closer to the array’s tail. this sums up to

   $$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

   so, $\Theta(n^2)$ — we already accounter for the ‘method1‘’s loop itself on this calculation.

   and lastly, on (5), this just swaps the ‘index‘ entry of the array with the greatest/ largest value in the remaining range, as ‘helperMethod1‘ returns to ‘marker‘ in (4). the ‘swap‘ method only cost 3 operations, so $\Theta(1)$

   so, this ‘method1‘ method got a time complexity of $\boxed{\Theta(n^2)}$, for both best case and worse case as ‘helperMethod1‘ goes through every entry in the given range anyway, and I think this is selection sort?

---

2. the code iterate through the array of integer, ‘array‘, and checks whether ‘key‘ exists inside the array or not, with early exit in case ‘key‘ was found early. So, best case, ‘array[0]‘ == ‘key‘, and the method returns true — that’s $\boxed{\Theta(1)}$; and for worse case: ‘key‘ does not exist in ‘array‘ or ‘key’ == ‘array[n-1]‘ (last index of the array) — this will resulted in $\boxed{\Theta(n)}$

---

3. as there's no conditions or early exits, we just compute the complexity of each layers, and as it's sequencial, we use product rule.

   - in (4), this is quite dependent layer, as it doesn't affect (5), (6), or the operation inside this triple-nested loop itself. it just goes through

     $$100, 99, 98, ..., 6, 5, 4$$

     which is a multiplication factor of 97 to our complexity — still it is $\Theta(1)$
   - for (5), this loops through

     $$0, 1, 2, 3, ..., 2n-2, 2n-1$$

     thats $2n$ operations — $\Theta(n)$, which will be multiplied to our final complexity.
   - lastly, (6), this one is a logarithmic scale, as 'count' starts at $4n$, and got reduced by half in each iteration, as

     $$4n, 2n, n, \frac{n}{2}, \frac{n}{4}, ..., 4, 2, 1$$

     this'd cost $\log_2 4n \in \Theta(\log_2 n)$

   and for the in-loop operation, 'sum += 1.0 * array[index/2]/count;': addition, search, multiplication and two divisions; supposed all took 1 operations, so that's 5 operations — $\Theta(1)$.

   we combined all these time complexity as

   $$97 \cdot 2n \cdot \log_2 4n \cdot 5 = cn\log_2 4n \in \boxed{\Theta(n \log_2 n)}$$

   this applies for both best case and worse case, as there's nothing in this method that could affect the running time, but the length of 'array'.

Task 6: Recursive Code (6 points)

For each of the following Java functions:

1. Describe how you will measure the problem size in terms the input parameters. For example, the input size is measured by the variable n, or the input size is measured by the length of array a.
2. Write a recurrence relation representing its running time. Show how you obtain the recurrence.
3. Indicate what your recurrence solves to (by looking up the recurrence in our table).

```
// assume xs.length is a power of 2
int halvingSum(int[] xs) {
    if (xs.length == 1) return xs[0];
    else {
        int[] ys = new int[xs.length/2];
        for (int i=0;i<ys.length;i++)
```

```
            ys[i] = xs[2*i]+xs[2*i+1];
        return halvingSum(ys);
    }
}

int anotherSum(int[] xs) {
    if (xs.length == 1) return xs[0];
    else {
        int[] ys = Arrays.copyOfRange(xs, 1, xs.length);
        return xs[0]+anotherSum(ys);
    }
}

int[] prefixSum(int[] xs) {
    if (xs.length == 1) return xs;
    else {
        int n = xs.length;
        int[] left = Arrays.copyOfRange(xs, 0, n/2);
        left = prefixSum(left);
        int[] right = Arrays.copyOfRange(xs, n/2, n);
        right = prefixSum(right);
        int[] ps = new int[xs.length];
        int halfSum = left[left.length-1];
        for (int i=0;i<n/2;i++) { ps[i] = left[i]; }
        for (int i=n/2;i<n;i++) { ps[i] = right[i - n/2] + halfSum; }
        return ps;
    }
}
```

Solution

we'll measure 'halvingSum''s problem size by array's length of 'xs', which

$$\boxed{n = \text{xs.length}}$$

for base case, the method will terminates when 'xs.length' equals to 1, so

$$T(1) = \Theta(1)$$

and in each recursives,

- allocate 'ys' with half the length of 'xs.length', so 'ys''s length is $\frac{n}{2}$ — that's $\Theta(n)$
- the loop goes through the range of 'ys' which is $(0, \frac{n}{2})$ — so $\Theta(n)$
- in-loop operation do access 2 indexes of 'xs', adds them, and write to a cooresponding index in 'ys' — that's 4 operations, so $\Theta(1)$

so, $\Theta(n)$ in total time complexity in each method call; which the method will recursively call itself, with 'ys' as it's argument. the recurrence could be written as

$$\boxed{T(n) = \Theta(n) + T(\frac{n}{2})}$$

and, we try to solve the recurrence, by writing out the first few terms, and the last term

$$T(n) = \Theta(n) + T(\frac{n}{2}) \quad \text{— } \Theta(n)$$
$$T(\frac{n}{2^1}) = \Theta(\frac{n}{2^1}) + T(\frac{n}{2^2}) \quad \text{— } \Theta(\frac{n}{2^1})$$
$$T(\frac{n}{2^2}) = \Theta(\frac{n}{2^2}) + T(\frac{n}{2^3}) \quad \text{— } \Theta(\frac{n}{2^2})$$
$$...$$
$$T(\frac{n}{2^k}) = T(1) = \Theta(\frac{n}{2^k}) + T(\frac{n}{2^{k+1}}) = \Theta(1) + T(0) \quad \text{— } \Theta(\frac{n}{2^k}) = \Theta(1)$$

so, the combined complexity is

$$\Theta(n + \frac{n}{2} + \frac{n}{4} + ... + 2^1 + 1) \leq \Theta(2n)$$

therefore, $\boxed{T(n) \in \Theta(n)}$ — as ‘n‘ decrease in every call, and our recurrence already write it out well (i guess), the method that gets called don't split, so we don't have to do further conversion from exponential to linear scale (unlike the usual).

‘anotherSum‘ canbe measured by the length of ‘xs‘, which is

$$\boxed{n = \text{xs.length}}$$

for base case, the method will terminate when ‘xs.length‘ equals to 1, so

$$T(1) = \Theta(1)$$

and in each recursives,

- ‘xs‘ got copied from index 1 to the end (skip index 0), to ‘ys‘, $n - 1$ operations — so $\Theta(n)$
- the method call itself, with the argument of the newly copied array with length (n-1)
- on the same line, there's addition — $\Theta(1)$

so, $\Theta(n)$ in total time complexity in each method call; which the method will recursively call itself, with ‘ys‘ as it's arguemnt, the recurrence could be written as

$$\boxed{T(n) = T(n-1) + \Theta(n)}$$

and, we try to solve the recurrence, by writing out the first few terms, and the last term

$$T(n) = \Theta(n) + T(n-1) \quad \text{— } \Theta(n)$$
$$T(n-1) = \Theta(n-1) + T(n-2) \quad \text{— } \Theta(n-1) \equiv \Theta(n)$$
$$T(n-2) = \Theta(n-2) + T(n-3) \quad \text{— } \Theta(n-2) \equiv \Theta(n)$$
$$...$$
$$T(1) = \Theta(1) + T(0) \quad \text{— } \Theta(1)$$

so, the combined complexity is

$$\Theta(n + (n-1) + (n-2) + (n-3) + ... + 2 + 1)$$

which is equivalent to

$$\Theta(n^2 - \frac{(n-1)(n)}{2}) \in \boxed{\Theta(n^2)}$$

---

we'll measure 'prefixSum' problem size by array's length of 'xs', which

$$n = \text{xs.length}$$

for base case, the method will terminates when 'xs.length' equals to 1, so

$$T(1) = \Theta(1)$$

and in each recursives

- get 'n''s length — $\Theta(1)$
- copy the left half, $\frac{n}{2}$ operations — that's $\Theta(n)$
- the method call itself with that left half, of size $\frac{n}{2}$
- copy the right half, $\frac{n}{2}$ operations — that's $\Theta(n)$
- the method call itself with that right half, of size $\frac{n}{2}$
- allocate the array 'ps' of size $\frac{n}{2}$ — so $\Theta(n)$
- access the last element of the left half — $\Theta(1)$
- loop through the left half (I supposed the in-loop operations here doesn't matter now), $\frac{n}{2}$ operations — $\Theta(n)$
- same, loop on the right half, $\frac{n}{2}$ operations — $\Theta(n)$

we'd ignore some of the step above, as it's too detailed, only focus on $\Theta(n)$ ones, the time complexity wouuld be $\Theta(\frac{5}{2}n)$ in total for each method call; which the method will recursively call itself, with the left and right half of the initial array as an argument, the recurrence could be written as

$$\boxed{T(n) = 2T(\frac{n}{2}) + \Theta(\frac{5}{2}n))}$$

and, we try to solve the recurrence, by writing out the first few terms, and the last term (just like (1)) — I simplify the time complaxity, as the rough complexity should remain the same (it's easier to calculate without fractions)

$$T(n) = \Theta(n) + 2T(\frac{n}{2}) \quad \text{— } \Theta(n)$$

$$T(\frac{n}{2^1}) = \Theta(\frac{n}{2^1}) + 2T(\frac{n}{2^2}) \quad \text{— } 2^1 \cdot \Theta(\frac{n}{2^1}) = \Theta(n)$$

$$T(\frac{n}{2^2}) = \Theta(\frac{n}{2^2}) + 2T(\frac{n}{2^3}) \quad \text{— } 2^2 \cdot \Theta(\frac{n}{2^2}) = \Theta(n)$$

...

$$T(\frac{n}{2^k}) = \Theta(\frac{n}{2^k}) + 2T(\frac{n}{2^{k+1}}) \quad — 2^k \cdot \Theta(\frac{n}{2^k}) = \Theta(n)$$

the recursion stops when

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$

so, the combined complexity is

$$\Theta(\underbrace{n + n + n + ... + n}_{k})$$

therefore,

$$\boxed{T(n) \in \Theta(n \log n)}$$

Task 7: Counting Dashes (2 points)

Consider the following program:

```
void printRuler(int n) {
    if (n > 0) {
        printRuler(n-1);
        // print n dashes
        for (int i=0;i<n;i++) System.out.print('-');
        System.out.println();
        // -------------
        printRuler(n-1);
    }
}
```

We would like to know the total number of dashes printed for a given n. If we are to write a recurrence for that, we will get

$$g(n) = 2g(n-1) + n, \quad g(0) = 0,$$

where the additive n term stems from the fact that we print exactly n dashes in that function call.
It may seem hopeless to try to solve this recurrence directly, but you may recall that the number of instructions taken to solve tower of Hanoi on n discs is given by the recurrence

$$f(n) = 2f(n-1) + 1, \quad f(0) = 0.$$

As you showed in a previous assignment, $f(n)$ has a closed-form of $f(n) = 2^n - 1$.
The two recurrences are strikingly similar. In this problem, we'll analyze $g(n)$ using our knowledge of $f(n)$. Since $f(n)$ and $g(n)$ have similar recurrences, differing only in an n term, we're going to guess that

$$g(n) = a \cdot f(n) + b \cdot n + c. \tag{1}$$

The following steps will guide you through determining the values of a, b, and c—and verifying that our guess indeed works out. In your writeup, clearly show your work.

(i) We'll first figure out the value of c. What do you get when plugging in $n = 0$ into equation (1)? It helps to remember that $f(0) = g(0) = 0$. (Hint: c should be 0.)

(ii) To figure out the values of a and b, we'll plug in $g(n)$ from equation (1) into the recurrence $g(n) = 2g(n-1) + n$. You should be able write it as $(\cdot)n + (\cdot) = 0$ and solve for a and b such that both coefficients are zero. Keep in mind: Because $f(n) = 2f(n-1)+1$, we know that $f(n) - 2f(n-1) = 1$.

(iii) Derive a closed form for $g(n)$.

(iv) Use induction to verify that your closed form for $g(n)$ actually works.

Solution

(i) by plugging $n = 0$ into (1), we get

$$\begin{aligned} g(0) &= a \cdot f(0) + b \cdot 0 + c \\ 0 &= a(0) + b(0) + c \\ 0 &= 0 + 0 + c \end{aligned}$$

therefore, $\boxed{c = 0}$.

(ii) plugging in $g(n)$ from (1), into the recurrence $g(n) = 2g(n-1) + n$, we get

$$a \cdot f(n) + b \cdot n = 2\left(a \cdot f(n-1) + b \cdot (n-1)\right) + n$$

$$af(n) + bn = 2af(n-1) + 2b(n-1) + n$$

format this as

$$af(n) - 2af(n-1) + bn - 2b(n-1) - n = 0$$

$$a(f(n) - 2f(n-1)) + (bn - 2bn + 2b) - n = 0$$

$$a(f(n) - 2f(n-1)) + (-bn + 2b) - n = 0$$

and as hinted that $f(n) = 2f(n-1) + 1 \Rightarrow f(n) - 2f(n-1) = 1$, we could derive

$$a - bn + 2b - n = 0$$

rearranging as

$$(-b-1)n + (a + 2b) = 0$$

which means that

$$-b - 1 = 0 \tag{i}$$

$$a + 2b = 0 \tag{ii}$$

from (i), we get $\boxed{b = -1}$, and substituting into (ii),

$$a + 2(-1) = 0 \Rightarrow \boxed{a = 2}.$$

(iii) therefore,

$$\boxed{g(n) = 2f(n) - n}$$

since $f(n) = 2^n - 1$, as hinted (tower of hanoi), we got

$$g(n) = 2(2^n - 1) - n$$

$$\boxed{g(n) = 2^{n+1} - n - 2}$$

(iv) *Proof.* we claim that

$$P(n) \equiv g(n) = 2^{n+1} - n - 2$$

for the **base case** where $n_0 = 0$

$$\begin{aligned} P(0) &= g(0) = 0 = 2^{0+1} - 0 - 2 \\ P(1) &= g(1) = 1 = 2^{1+1} - 1 - 2 \\ P(2) &= g(2) = 4 = 2^{2+1} - 2 - 2 \end{aligned}$$

then, we assume P(k) for arbitrary ($k \geq n_0$), we prove $p(k+1) \equiv g(k+1)2^{(k+1)+1} - (k+1) - 2$

$$\begin{aligned} g(k+1) &= 2g(k) + (k+1) && \text{recurrence form of } g(n) \\ &= 2\big(2^{k+1} - k - 2\big) + (k+1) && \text{inductive hypothesis} \\ &= 2^{k+2} - 2k - 4 + k + 1 \\ &= 2^{k+2} - k - 3 \\ &= 2^{(k+1)+1} - (k+1) - 2 && \text{format until satisfied } P(k+1) \end{aligned}$$

since P(k+1) follows P(k), the **induction case** is satisfied, thus we can conclude the given relation is true $\square$